

프론트엔드 프롬프트 구조 (Vue3 + Vite + Vue Router + Pinia)

Plan 모드에 입력 → 질문에 답하며 계획 다듬기 → "구현 시작"으로 Agent 실행. 순수 질문은 Ask 모드.

아래 프롬프트에서 (여기에 적으세요) 라고 된 부분만 자유롭게 채우세요. 나머지 줄은 그대로 복사해서 쓰면 됩니다. 기술적인 부분은 이미 정해져 있어서, 뭘 쓸지 판단하지 않아도 됩니다.

작업 순서

- [1단계: 프로젝트 구조 만들기] 프롬프트를 Plan 모드에 입력합니다.
- 질문에 답하고 "구현 시작"으로 Agent가 만들게 합니다.
- 기능을 하나 정합니다.
- 그 기능을 위한 [2단계: 화면 자리 만들기] 프롬프트를 Plan 모드에 입력합니다.
- [3단계: 화면 내용 채우기] 프롬프트를 Plan 모드에 입력하고 같은 방식으로 반복합니다.
- 순수 질문(용어, 예러 의미 등)은 Ask 모드로.

1단계: 프로젝트 구조 만들기 (스캐폴드)

구조

[지금 상황]

- (여기에 적으세요: 프로젝트가 있는지 없는지)

[부탁하고 싶은 것]

질문 없이 바로 생성되는 방식으로 **vue3+vite** 프로젝트를 만들어줘. 화면 이동 기능과 데이터 공유 기능도 같이 넣어줘.

이런 화면을 만들 거야: (여기에 적으세요: 어떤 화면인지, 뭐가 있어야 하는지)

[지켜야 할 것]

- 폴더는 재사용할 화면 요소 폴더, 화면 페이지 폴더, 화면 이동 설정 폴더, 데이터 공유 설정 폴더로 나눠줘.
- 대화형 질문이 뜨는 도구 말고, 질문 없이 바로 생성되는 명령어를 사용해줘.
- 설치 명령어(**package.json**이 있는 폴더에서 **npm i** 실행), 실행 명령어(**npm run dev**), 확인 방법을 **README.md**에 정리해줘.
- 지금 당장 채우지 않고 나중에 채울 빈 파일에는 **TODO: 무엇을 해야 하는지 - 왜 필요한지**로 표시해줘. 기본으로 만들어지는 파일(이미 동작하는 파일)에는 **TODO**를 넣지 마.

[확인 방법]

- **README.md**에 적힌 대로 그대로 따라 했을 때, 에러 없이 화면이 뜨는지 확인해줘.

예시

[지금 상황]

- 프로젝트 자체가 아직 없다. 빈 폴더만 있는 상태다.

[부탁하고 싶은 것]

질문 없이 바로 생성되는 방식으로 **Vue3+vite** 프로젝트를 만들어줘. 화면 이동 기능과 데이터 공유 기능도 같이 넣어줘.

이런 화면을 만들 거야: 할 일을 추가하고, 목록으로 보고, 완료 표시하고, 지울 수 있는 화면.

[지켜야 할 것]

- 폴더는 재사용할 화면 요소 폴더, 화면 페이지 폴더, 화면 이동 설정 폴더, 데이터 공유 설정 폴더로 나눠줘.
- 대화형 질문이 뜨는 도구 말고, 질문 없이 바로 생성되는 명령어를 사용해줘.
- 설치 명령어(**package.json**이 있는 폴더에서 **npm i** 실행), 실행 명령어(**npm run dev**), 확인 방법을 **README.md**에 정리해줘.
- 나중에 채울 빈 파일들의 이름만 미리 만들어줘: 화면 전체 파일, 입력창 파일, 목록 파일, 항목 하나짜리 파일.
- 각 빈 파일 상단에 **TODO:** 무엇을 해야 하는지 - 왜 필요한지 로 남겨줘. 기본으로 만들어지는 파일에는 **TODO**를 넣지 마.

[확인 방법]

- **README.md**에 적힌 대로 그대로 따라 했을 때, 에러 없이 기본 화면이 뜬다.
- 요청한 빈 파일들이 만들어져 있다.

2단계: 화면 자리 만들기 (아직 안 보이는 부분)

새로운 기능을 시작할 때 가장 먼저 쓰는 단계입니다. 새 화면이 필요한지, 여러 화면 요소가 데이터를 같이 써야 하는지는 몰라도 됩니다. 어떤 기능을 원하는지만 설명하면, 필요한 걸 AI가 알아서 판단해서 자리만 만들어줍니다. **이 단계가 끝나도 화면에는 아직 아무것도 안 보입니다.** 실제로 보이는 걸 만드는 건 다음 3단계입니다.

구조

[추가하고 싶은 기능]

- (여기에 적으세요: 어떤 기능을 추가하고 싶은지)

[지금 상황]

- 프로젝트 구조는 이미 만들어져 있음 (1단계에서 생성)
- (여기에 적으세요: 관련된 기존 코드가 있으면 붙여넣기)

[지켜야 할 것]

- 이 기능에 새 화면이 필요한지, 여러 화면 요소가 같이 쓸 데이터가 필요한지는 네가 판단해서 알려주고, 그에 맞게 만들어줘.
- 지금 당장 채우지 않는 부분은 **TODO:** 무엇을 해야 하는지 - 왜 필요한지 로 남겨줘.

[확인 방법]

- (여기에 적으세요: 어떻게 동작해야 하는지)

예시

[추가하고 싶은 기능]

- 할 일 목록을 보여주는 화면. 여기 있는 할 일 데이터는 다른 화면에서도 같이 써야 한다.

[지금 상황]

- 프로젝트 구조는 이미 만들어져 있음 (1단계에서 생성)

[지켜야 할 것]

- 이 기능에 새 화면이 필요한지, 여러 화면 요소가 같이 쓸 데이터가 필요한지는 네가 판단해서 알려주고, 그에 맞게 만들어줘.
- 지금 당장 채우지 않는 부분은 **TODO**: 무엇을 해야 하는지 - 왜 필요한지 로 남겨줘.

[확인 방법]

- 할 일 목록 화면으로 이동이 되고, 데이터가 잘 공유되는지 콘솔로 확인할 수 있다.

3단계: 화면 내용 채우기 (실제로 보이는 부분, 기능 하나씩 반복)

2단계에서 만든 자리 안에, 사용자가 실제로 보고 누르는 것(입력창, 버튼, 목록 등)을 하나씩 채워 넣는 단계입니다. 기능 하나에 필요한 요소 수만큼 여러 번 반복합니다.

구조

[만들 것]

- (여기에 적으세요: 어떤 기능인지, 화면에 뭐가 보여야 하는지)

[지금 상황]

- 프로젝트 구조는 이미 만들어져 있음
- (여기에 적으세요: 관련된 기존 코드가 있으면 붙여넣기)

[지켜야 할 것]

- 이 화면 요소가 다른 화면 요소나 저장소와 정보를 주고받아야 한다면, 그 방식은 네가 알아서 적절하게 설계해줘.
- 불러오는 중이거나 비어있거나 에러가 났을 때는 네가 알아서 자연스럽게 표시해줘.
- 미완성 부분은 **TODO**: 무엇을 해야 하는지 - 왜 필요한지 로 남겨줘.

[확인 방법]

- (여기에 적으세요: 눌러봤을 때 어떻게 동작해야 하는지)
- 파일 하나로 줘.

예시

[만들 것]

- 할 일을 추가하는 입력창. 텍스트 입력창 1개 + 추가 버튼 1개.
- 입력창이 비어있으면 버튼을 눌러도 아무 일도 일어나지 않는다.

[지금 상황]

- 프로젝트 구조는 이미 만들어져 있음. 입력창을 담당할 빈 파일도 이미 만들어져 있다.
- 할 일 목록 저장소도 이미 만들어져 있음 (2단계에서 생성).

[지켜야 할 것]

- 추가 버튼을 누르면 저장소에 새 할 일을 추가하도록 연결해줘.
- 미완성 부분은 **TODO**: 무엇을 해야 하는지 - 왜 필요한지 로 남겨줘.

[확인 방법]

- 입력 후 추가 버튼을 눌렀을 때, 저장소에 값이 정상적으로 추가되는지 확인해줘.
- 파일 하나로 줘.

다음 기능(목록 보기, 완료 표시, 삭제)도 같은 구조로 하나씩 반복합니다.

막힐 때: Ask 모드

Plan이나 Agent가 만든 코드를 고치려는 게 아니라, 그냥 궁금한 게 생겼을 때 쓰는 모드입니다. Ask 모드는 파일을 건드리지 않고 설명만 해주기 때문에, 지금 진행 중인 계획이나 코드에 실수로 영향을 주지 않습니다.

이럴 때 쓰면 됩니다.

- 코드에 나온 낯선 용어나 문법이 궁금할 때
- 에러 메시지가 무슨 뜻인지 궁금할 때
- 왜 이렇게 만들었는지 이유가 궁금할 때

방금 만든 코드에서 (여기에 적으세요: 궁금한 부분)이 정확히 뭘 의미하는지 쉽게 설명해줘.

이 에러 메시지가 무슨 뜻인지 쉽게 설명해줘: (여기에 적으세요: 에러 메시지 붙여넣기)

질문이 끝나면 다시 Plan 모드로 돌아가서 하던 작업을 이어가면 됩니다.

용어 정리

- **Plan 모드**: 코드를 바로 고치지 않고, 계획과 질문으로 먼저 정리해주는 모드
- **Agent 모드**: 파일을 직접 만들고 고치고, 터미널 명령까지 실행하는 가장 자율적인 모드
- **Ask 모드**: 코드를 고치지 않고 궁금한 것에 설명만 해주는 모드
- **스캐폴드**: 실제 기능을 채우기 전, 폴더와 파일만 미리 잡아둔 프로젝트 뼈대
- **Vue Router**: 화면(페이지)을 여러 개 만들고 그 사이를 이동할 수 있게 해주는 도구
- **Pinia**: 여러 화면이나 화면 요소가 같은 데이터를 함께 쓸 수 있게 해주는 저장소 도구
- **저장소(store)**: Pinia로 만든, 데이터를 저장하고 여러 곳에서 공유하는 창고 같은 파일
- **화면 요소**: 화면에 보이는 하나하나의 조각 (입력창, 버튼, 목록 등)
- **package.json**: 이 프로젝트에 어떤 패키지가 필요한지 적어놓은 설정 파일
- **npm i**: `npm install`의 줄임말. package.json에 적힌 패키지를 한 번에 설치하는 명령어 (package.json이 있는 폴더에서 실행해야 함)
- **README.md**: 프로젝트를 어떻게 설치하고 실행하는지 적어두는 안내 문서
- **TODO 주석**: 아직 끝나지 않은 부분을 코드 안에 표시해두는 방법